CornellBowers
College of Computing + Information Science

Reimplementation of LoRA: Low-Rank Adaptation of LLMs

Julian Gasharov
Cornell University

Introduction

- **Problem**
  Adapt a pre-trained LLM to multiple tasks by fine-tuning. Full fine-tuning is too expensive since it updates ALL the parameters.
- **Context and motivation**
  LLMs are too large: RoBERTa-Large has 355M params, GPT-3 has 175B. It is not feasible to store multiple copies of the model to full fine-tune for separate tasks.
- **Goal**
  Reduce the number of parameters used in fine-tuning, while keeping high accuracy
- **Results reproduced**
  For RoBERTa-Large with 355M params, reduce the number of params used in fine-tuning to about 0.8M, while keeping 96% validation accuracy.
- **Summary of LoRA [1]**
  LoRA is a parameter-efficient fine-tuning method for LLMs.

For a pretrained weight matrix $W \in \mathbf{R}^{d\times k}$, LoRA uses
$W + BA$ where $A \in \mathbf{R}^{r\times k}$, $B \in \mathbf{R}^{d\times r}$
with r much smaller than $\min(d, k)$, so low-rank.
During fine-tuning, W is frozen, while A, B contain the trainable parameters.

Instead of learning dk params, we learn r(d+k) params.

Methodology

- **Approach**
  Replace the layers in RoBERTa-Large with LoRA layers
  Models: RoBERTa-Large
  Datasets: GLUE
  Tools used: Adam optimizer, RoBERTa-Large, GLUE dataset, code in [2] for inspiration
- **Modifications and design choices**
  - used a lower rank (than in [1]) to update even fewer parameters
  - added a third matrix to increase accuracy

Results

- **Results & Benchmarks**

	type of fine-tuning	trainable params in fine-tuning	percentage of trainable params	validation accuracy
RoBERTa-large	full fine-tuning	355,755,010	100%	96.4%
results in [1]	LoRA rank 8	800,000	0.22%	96.2%
My results:				
[1] reimplemented	LoRA rank 4	395,266	0.11%	95.06%
with additional third matrix	LoRA rank 4	396,034	0.11%	95.87%
with additional third matrix	LoRA rank 8	791,554	0.22%	95.87%

Table1: Pre-trained model with 355,755,010 params is fine-tuned. Table shows the trainabale params and validation accuracy of the models. Higher is better for accuracy, and lower is better for params.

- **Challenges encountered during re-implementation**
  Converting the RoBERTa-Large layers to LoRA layers
- **Analysis of results in the context of [1]**
  - Used sst-2 training, validation, and testing, so the accuracy to be compared to, is the accuracy 96.2% in [1] on that metric.
  - Explored rank r=4 instead of r=8 in [1]. Due to lower rank, accuracy 95.06% is lower, but the advantage is half of the params used.
  - Explored adding an extra (third) matrix and rank r=4. It still uses half the params used for r=8, but improves accuracy to 95.87%, very close to 96.2% in [1].

Applications

With much fewer trainable params, it becomes feasible to fine-tune multiple versions of the pre-trained model on specific tasks.

Conclusion

LoRA can drastically reduce the number of parameters in fine-tuning while keeping high accuracy, allowing the model to be fine-tuned wit much lower storage and compute costs.

Directions for Future Work

- Understand why LoRA works
- Find methods how to pick the matrices, instead of using heuristics
- Combine LoRA with other efficient adaptation methods

cartoon generated by ChatGPT [3]

References

[1] LoRA: Low-Rank Adaptation of Large Language Models, Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, *International Conference on Learning Representations (ICLR) in 2022*, arXiv: 2106.09685.
[2] Code Repository on GitHub: github.com/microsoft/LoRA
[3] ChatGPT was used to produce the cartoon.